Sistemas Distribuidos

Práctica 1
Water Management Network

Rafael Peiró y Santiago Serrano
Universidad de Alicante
3ºIngeniería Informática

Índex

1.Introducción y Objetivos..... 4
2.Arquitectura del sistema..... 5
3.Tecnologías utilizadas..... 6
4.Protocolos de comunicación..... 7
4.1.Comunicación Monitor-Central..... 7
4.2.Comunicación Monitor-Engine..... 8
4.3.Eventos Kafka..... 8
4.4.Eventos dirigidos al FO..... 10
4.5.Identificadores y control de duplicados..... 10
5.Modelos de datos y persistencia..... 11
5.1.Alta, baja y reactivación..... 12
5.2 Estado local del Engine..... 12
5.3 Datos locales de FO..... 12
6.Estados de una estación..... 12
7.Flujo de funcionamiento..... 13
7.1.Arranque de Central..... 13
7.2.Registro de una estación..... 14
7.3.Arranque de operario y consulta de estaciones..... 14
7.4.Solicitud de riego desde FO..... 14
7.5.Durante el riego..... 15
7.6.Finalización del riego..... 15
7.7. Detección de averías y recuperación de Engine..... 16
7.8. Control desde Central y desde la estación..... 16
7.9. Ejecución de servicios desde fichero..... 17
7.10. Administración de identidades y consulta de logs..... 17
8.Despliegue previsto..... 18
8.1.Distribución de los componentes..... 18
8.2.Requisitos previos..... 18
8.3. Configuración de los módulos..... 19
8.4.Preparación de Kafka..... 19
9.Decisiones de diseño..... 20
9.1. Separación entre supervisión y operación..... 20
9.2. División de la estación en Monitor y Engine..... 20
9.3. Autorización desde Central..... 20
9.4. Identificación de solicitudes, sesiones y eventos..... 20
9.5. Reserva y confirmación de inicio..... 21
9.6. Eventos dirigidos a cada operario..... 21
9.7. Persistencia centralizada..... 21
9.8. Panel web..... 21
10. Resiliencia y recuperación..... 22
10.1. Criterio de recuperación..... 22
10.2. Caída de Central..... 22

10.3. Caída del Monitor.....22
10.4. Caída o avería del Engine.....22
10.6. Fallos de Kafka.....23
10.7. Duplicados y consistencia.....23
11.Conclusión.....24

1.Introducción y Objetivos

La práctica consiste en desarrollar un sistema distribuido que simula la gestión centralizada del riego de parques y jardines de una ciudad. El sistema permite supervisar estaciones de riego, solicitar servicios desde aplicaciones de operario, autorizar riegos, recibir telemetría en tiempo real, detectar incidencias y recuperar el servicio tras fallos de componentes.

El objetivo principal es aplicar conceptos de sistemas distribuidos mediante la combinación de dos mecanismos de comunicación:

- **Sockets TCP**, para comunicaciones directas, sincronas y de supervisión entre Monitor, Engine y Central.
- **Apache Kafka**, para desacoplar solicitudes, órdenes, telemetría, resultados y notificaciones dirigidas a los operarios.

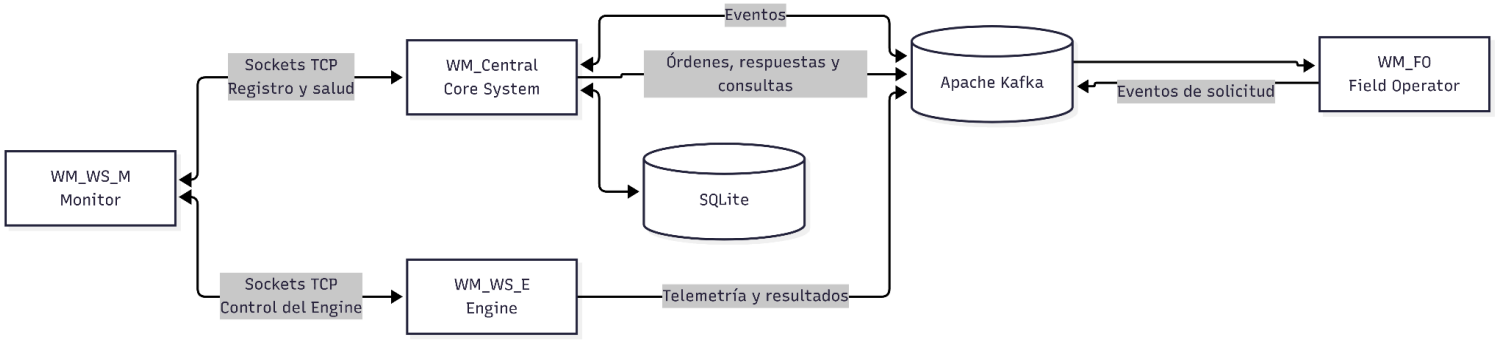
Los objetivos concretos son:

- Implementar una central de control capaz de registrar, monitorear y gobernar varias estaciones de riego.
- Permitir que los operarios soliciten riegos y reciban autorizaciones, denegaciones, telemetría y resultados finales.
- Simular el funcionamiento físico de cada estación mediante un Engine y un Monitor independientes.
- Mantener un panel de estado con información actualizada de cada estación.
- Garantizar que las estaciones no admitan riegos cuando estén desconectadas, averiadas, bloqueadas o regando.
- Detectar fugas o fallos del Engine y notificarlos a Central y al operario que toque.
- Implementar mecanismos básicos de resiliencia ante caídas de Monitor, Engine, FO o Central.
- Desplegar Central, SQLite y Kafka en la nube, y los módulos FO y WS en los equipos del laboratorio.

2.Arquitectura del sistema

El sistema se compone de cinco elementos principales:

Componente	Función
WM_Central	Núcleo de control. Registra estaciones, procesa solicitudes, autoriza riegos, consume telemetría, gestiona incidencias y ofrece el panel web.
WM_WS_M	Monitor de una estación. Se autentica en Central, supervisa la salud del Engine y notifica incidencias y recuperaciones.
WM_WS_E	Engine de una estación. Simula la electroválvula, el caudalímetro, el riego, el volumen acumulado y las averías.
WM_FO	Aplicación de operario. Solicita riegos y recibe únicamente los eventos relacionados con sus servicios.
SQLite	Base de datos de Central. Almacena estaciones, operarios, solicitudes, sesiones, telemetría, resultados e incidencias.
Kafka	Intercambia solicitudes, órdenes, telemetría y resultados entre los módulos mediante eventos.



- **Monitor** es el único componente que se comunica directamente con Central mediante sockets.
- **Engine** recibe órdenes de negocio por Kafka y responde con telemetría y resultados por Kafka.
- **Monitor** comprueba la salud del Engine cada segundo mediante un socket TCP.
- Cada operario dispone de un topic Kafka propio, de modo que solo recibe los eventos de los riegos que le corresponden.

Esta separación permite que el fallo de un componente no afecte a todo el sistema. Por ejemplo, si Central se cae, un riego ya iniciado puede continuar hasta finalizar; si el Monitor se cae, el Engine puede terminar el servicio en curso, pero no se admiten nuevos riegos.

3.Tecnologías utilizadas

Tecnología	Uso en el proyecto
Java 17	Lenguaje principal de todos los módulos.
Maven	Compilación, gestión de dependencias y generación del JAR ejecutable.
Sockets TCP	Comunicación Monitor–Central y Monitor–Engine.
Apache Kafka 3.9	Streaming de eventos entre FO, Central y Engine.
SQLite	Persistencia de estaciones, operarios, solicitudes, sesiones, telemetría e incidencias.
Jackson	Serialización y validación de mensajes JSON.
Docker	Empaquetado y despliegue de los módulos.
Docker Compose	Manejo local de Kafka, Central, Monitores, Engines y FO.
Railway	Despliegue en la nube de Central, SQLite y Kafka.
HTTP + HTML/CSS/JavaScript	Panel web de monitorización de Central, web de operario, y web de dar de alta.

Se usa kafka para desacoplar los módulos, ya que un FO no conoce directamente al Engine, y Central no necesita mantener conexiones permanentes con cada operario. Los sockets se usan para las comprobaciones de salud y el registro, ya que aquí es necesario conocer el estado actual del módulo.

4.Protocolos de comunicación

4.1.Comunicación Monitor-Central

El Monitor abre una conexión TCP con Central y envía tramas de texto con campos separados por #.

Mensaje	Sentido	Formato
REGISTER	Monitor → Central	REGISTER#messageId#stationId
REGISTER_RESULT	Central → Monitor	REGISTER_RESULT#correlationId#OK/ERROR#errorCode#message
MONITOR_HEARTBEAT	Monitor → Central	MONITOR_HEARTBEAT#messageId#stationId#engineHealth#occurredAt
INCIDENT	Monitor → Central	INCIDENT#messageId#stationId#cause#sessionId#occurredAt
RECOVERY	Monitor → Central	RECOVERY#messageId#stationId#occurredAt
ERROR	Cualquiera	ERROR#correlationId#errorCode#message

Ejemplo de registro correcto:

- REGISTER#550e8400-e29b#WS-01
- REGISTER_RESULT#550e8400-e29b#OK##Estación registrada correctamente

Ejemplo de incidencia:

- INCIDENT#a1b2c3#WS-01#ENGINE_KO#session-01#2026-11-01T17:00:00Z

Central solo acepta estaciones registradas y activas, y rechaza otro Monitor conectado con el mismo ID. La ubicación se administra en SQLite y el Monitor no la modifica durante el registro.

Los heartbeats correctos no necesitan respuesta. Central aplica un timeout de lectura y marca la desconexión al perder la comunicación. La consola omite impresiones rutinarias de heartbeat, pero conserva transiciones, errores, conexiones e incidencias.

Si deja de recibir mensajes del Monitor durante el timeout configurado, marca la estación como DISCONNECTED.

4.2.Comunicación Monitor-Engine

El Monitor abre un servidor TCP y espera la conexión del Engine:

Mensaje	Sentido	Formato
ENGINE_HELLO	Engine → Monitor	ENGINE_HELLO#messageId#stationId
ENGINE_HELLO_RESULT	Monitor → Engine	ENGINE_HELLO_RESULT#correlationId#OK/ERROR#errorCode
HEALTH_CHECK	Monitor → Engine	HEALTH_CHECK#messageId#occurredAt
HEALTH_RESULT	Engine → Monitor	HEALTH_RESULT#correlationId#OK/KO#sessionId
ERROR	Cualquiera	ERROR#correlationId#errorCode#message

Ejemplo:

- ENGINE_HELLO#msg-10#WS-01
- ENGINE_HELLO_RESULT#msg-10#OK#
- HEALTH_CHECK#msg-11#2026-11-01T17:00:00Z
- HEALTH_RESULT#msg-11#OK#session-01

Si el Engine responde KO o no responde antes del timeout, el Monitor envía un INCIDENT a Central. Cuando vuelve a responder OK, envía un RECOVERY.

Las órdenes de riego no van por este socket: el Engine las recibe por Kafka. Así se separa la supervisión de salud de la lógica de negocio.

Engine conecta con Monitor y presenta el ID de la estación. Monitor acepta solo el ID esperado y una vez autorizado por Central. Después envía comprobaciones periódicas.

4.3.Eventos Kafka

Los mensajes Kafka se envían en JSON UTF-8. Todos incluyen al menos:

```
{
  "schemaVersion": 1,
  "eventId": "uuid",
  "eventType": "TIPO_DE_EVENTO",
  "occurredAt": "2026-11-01T17:00:00Z"
}
```

Topic	Productor	Consumidor	Uso
wm.watering.requests	FO, Central o Engine	Central	Solicitudes de riego.
wm.station.commands	Central	Engine	Órdenes de inicio, parada, bloqueo y activación.
wm.station.telemetry	Engine	Central	Caudal y volumen durante el riego.
wm.station.results	Engine	Central	Confirmación de inicio y resultado final.
wm.fo.<operatorId>.events	Central	FO correspondiente	Respuestas y eventos dirigidos a un operario.
wm.fo.queries	FO	Central	Consulta de estaciones disponibles.

Cada estación usa un grupo de consumo distinto para recibir las órdenes y filtra el contenido por stationId. No se utiliza un único grupo compartido entre todas las estaciones, cada operario consume su topic específico.

Central procesa normalmente un registro por lote. Los mensajes inválidos pueden descartarse con un mensaje de error; confirmar el offset de un mensaje descartado es una decisión explícita y no equivale a ejecutar una operación válida.

Ejemplo de solicitud:

```
{
  "schemaVersion": 1,
  "eventId": "event-01",
  "eventType": "WATERING_REQUESTED",
  "occurredAt": "2026-11-01T17:00:00Z",
  "requestId": "request-01",
  "stationId": "WS-01",
  "operatorId": "OP-01",
  "durationSeconds": 120
}
```

Ejemplo de orden:

```
{
  "schemaVersion": 1,
  "eventId": "event-02",
  "eventType": "START_WATERING",
  "occurredAt": "2026-11-01T17:00:01Z",
```

```
  "requestId": "request-01",
  "sessionId": "session-01",
  "stationId": "WS-01",
  "operatorId": "OP-01",
  "durationSeconds": 120
}
```


Ejemplo de telemetría:


```
{
  "schemaVersion": 1,
  "eventId": "event-03",
  "eventType": "WATERING_TELEMETRY",
  "occurredAt": "2026-11-01T17:00:02Z",
  "requestId": "request-01",
  "sessionId": "session-01",
  "stationId": "WS-01",
  "operatorId": "OP-01",
  "sequence": 1,
  "flowLitresPerMinute": 12.5,
  "volumeLitres": 0.21
}
```


4.4.Eventos dirigidos al FO

- WATERING_AUTHORIZED comunica la reserva
- WATERING_DENIED la denegación
- WATERING_STARTED confirma el inicio
- WATERING_TELEMETRY informa del progreso
- WATERING_FINISHED contiene el resultado
- STATIONS_RESULT responde a una consulta de disponibles
- QUERY_DENIED a una consulta no autorizada.
- INCIDENT comunica incidencia

INCIDENT no cierra la sesión ni sustituye el resumen recibido si Engine cae, el aviso puede llegar antes del resultado recuperado.

4.5.Identificadores y control de duplicados

- requestId: identifica una solicitud durante todo su ciclo de vida.
- sessionId: identifica un riego autorizado.
- eventId: identifica un evento Kafka concreto.
- messageId: identifica una trama de socket.

Central almacena las solicitudes procesadas y comprueba el requestId antes de crear una nueva sesión. De esta forma, una solicitud repetida no genera un segundo riego. Los resultados finales se guardan una sola vez por sessionId.

Engine conserva las sesiones ejecutadas en su estado persistente tras la corrección aplicada. Un duplicado de una sesión terminada no debe abrir otra vez la válvula

5.Modelos de datos y persistencia

La base de datos SQLite contiene las siguientes tablas principales:

Tabla	Función
operators	Operarios registrados en el sistema.
stations	Estaciones de riego y su ubicación.
station_runtime_state	Estado operativo actual de cada estación.
watering_requests	Solicitudes de riego y su decisión.
watering_sessions	Sesiones de riego autorizadas, en curso o finalizadas.
telemetry_samples	Muestras de caudal y volumen recibidas durante un riego.
watering_results	Resumen final de cada riego.
station_incidents	Incidencias abiertas y resueltas.
station_commands	Órdenes enviadas a las estaciones.
processed_events	Eventos Kafka ya procesados.
socket_messages	Registro de mensajes de socket.
operator_events_outbox	Notificaciones pendientes o publicadas hacia los operarios.

El estado visible de una estación se calcula a partir de:

- Si el Monitor está conectado.
- Si la estación está bloqueada.
- La salud comunicada por el Engine.
- Si existe una sesión de riego activa.
- Si existe una incidencia abierta.

DataBaseManager tiene el código SQL y sincroniza sus métodos públicos sobre una conexión compartida. La reserva de estación y la creación de solicitud/sesión se realizan en una transacción.

5.1.Alta, baja y reactivación

Dar de alta crea o reactiva el ID de la estación o el operario que indiques, actualizando su nombre o ubicación. Dar de baja establece active=0, conservando historial y claves foráneas por si tenemos que sacar datos de las estaciones que ya no estén de alta.

Se rechaza la baja si existe una sesión autorizada o en curso, para estaciones también se tiene que estar Monitor desconectado. La baja no detiene contenedores remotos ni equivale a un bloquear una estación. El bloqueo impide riegos pero conserva el alta. Reactivar el ID no quita un bloqueo anterior. El panel operativo filtra estaciones activas; el listado administrativo incluye activos e inactivos.

5.2 Estado local del Engine

Engine guarda en /data/engine-.json la sesión, contadores, duración objetivo, estado de bloqueo/avería, resultado pendiente y sesiones ejecutadas. Se escribe al aceptar una sesión, en los pasos simulados y en las transiciones relevantes. La sustitución utiliza un archivo temporal y movimiento atómico; los fallos de escritura se tratan como motivo para no continuar suministrando sin persistencia.

Al recuperar un estado que indicaba riego activo, se mantiene la válvula cerrada y se prepara el resumen interrumpido con los últimos valores guardados. No se reanuda automáticamente el tiempo restante. El resultado pendiente se publica cuando Kafka está disponible. Esta recuperación requiere conservar el volumen y el mismo stationId.

5.3 Datos locales de FO

FO monta /app/archivos_fo para su historial y archivos de solicitudes. Conserva los últimos 100 logs. Al reiniciar con el mismo ID, grupo y volumen, combina historial y eventos pendientes según las posiciones conservadas por Kafka.

6.Estados de una estación

Estado	Significado	Color en el panel
AVAILABLE	Monitor y Engine funcionan; no hay riego activo.	Verde
WATERING	Hay un riego autorizado y el Engine ha confirmado su inicio.	Verde parpadeante
LEAK	El Engine está averiado, no responde o se ha detectado una fuga.	Rojo.

OUT_OF_SERVICE	Central ha bloqueado deliberadamente la estación.	Naranja.
DISCONNECTED	Central no recibe mensajes de vida del Monitor.	Gris.

La prioridad de estados es:

1. DISCONNECTED, si el Monitor no está conectado.
2. OUT_OF_SERVICE, si Central ha bloqueado la estación.
3. LEAK, si el Engine está averiado o no responde.
4. WATERING, si existe una sesión activa confirmada por el Engine.
5. AVAILABLE, en cualquier otro caso.

Una reserva AUTHORIZED impide otra solicitud aunque todavía no se muestre WATERING

Al resolver una avería o terminar un riego se recalcula el estado; no se fuerza siempre AVAILABLE. Una estación bloqueada conserva OUT_OF_SERVICE y una desconectada conserva DISCONNECTED. Al activar una estación cuyo Engine sigue KO debe quedar LEAK.

Durante un riego, el panel muestra:
- ID de la estación.
- Ubicación.
- Estado.
- Caudal actual en litros por minuto.
- Volumen acumulado en litros.
- Identificador del operario que solicitó el riego.

7.Flujo de funcionamiento

7.1.Arranque de Central

1. Central arranca y lee su configuración: puerto de sockets, dirección de Kafka, puerto web y ruta de la base de datos.
2. Abre SQLite y crea las tablas e inserta los datos iniciales si todavía no existen.
3. Consulta las estaciones registradas y activas.
4. Marca inicialmente las estaciones como DISCONNECTED, porque todavía no conoce su estado real.
5. Prepara el servidor de sockets para recibir conexiones de los Monitores.
6. Inicia el procesamiento de solicitudes, consultas, telemetría y resultados mediante Kafka.
7. Inicia el panel web de monitorización y control.
8. Queda a la espera de conexiones y peticiones, atendiendo las comunicaciones de forma concurrente.

7.2.Registro de una estación

1. El Monitor se conecta por socket a Central.
2. Envía REGISTER con el identificador de la estación y el identificador del mensaje.
3. Central comprueba que la estación existe y está activa en SQLite.
4. Comprueba también que no tenga otro Monitor conectado con el mismo ID.
5. Si las comprobaciones son correctas, responde con REGISTER_RESULT indicando aceptación.
6. Si alguna comprobación falla, responde con el código de error.
7. Central marca el Monitor como conectado y recalcula el estado visible de la estación.
8. El Monitor confirma que está autorizado y preparado mediante el mensaje de consola MONITOR_READY.
9. Al desplegar Engine se conecta al Monitor y envía ENGINE_HELLO.
10. El Monitor valida que el Engine utiliza el ID esperado y acepta su conexión.
11. El Monitor envía comprobaciones HEALTH_CHECK y recibe respuestas HEALTH_RESULT.
12. El Monitor comunica su estado y la salud del Engine a Central mediante heartbeats.
13. La estación pasa a AVAILABLE cuando su salud está confirmada y no hay bloqueo ni otra condición que impida prestar servicio.

7.3.Arranque de operario y consulta de estaciones

1. El FO recibe la dirección de Kafka y su identificador de operario.
2. Consulta a Central por HTTP para comprobar que el ID existe y está activo.
3. Si el operario no está registrado o está inactivo, muestra un error y termina.
4. Si Central no está accesible, informa del fallo de conexión sin interpretarlo como un ID desconocido.
5. Si la validación es correcta, comprueba o crea su topic wm.fo.<operatorId>.events.
6. Prepara el productor y el consumidor Kafka.
7. Recupera el historial local disponible y arranca su interfaz web.
8. Cuando el usuario solicita consultar las estaciones, publica STATIONS_QUERY en wm.fo.queries.
9. Central valida el operario y consulta las estaciones disponibles.
10. Publica STATIONS_RESULT en el topic de ese operario, o QUERY_DENIED si la consulta no está autorizada.
11. El FO muestra la respuesta en su ventana de mensajes.

7.4.Solicitud de riego desde FO

1. El operario introduce el ID de la estación y la duración del riego.
2. El FO comprueba que los datos básicos sean válidos.
3. Genera un requestId y construye el evento WATERING_REQUESTED.
4. Lo publica en wm.watering.requests con clave stationId.
5. Central consume la solicitud y valida su formato.
6. Comprueba si el requestId ya corresponde a una solicitud registrada, para evitar crear otra sesión.
7. Comprueba que el operario y la estación existen y están activos.

8. Comprueba conexión del Monitor, salud del Engine, bloqueo, incidencias y ausencia de una reserva incompatible.
9. Si no puede autorizarla, publica WATERING_DENIED con el motivo de denegación.
10. Si puede autorizarla, reserva la estación y crea la solicitud y la sesión en una transacción de SQLite.
11. Publica WATERING_AUTHORIZED al FO.
12. Publica START_WATERING en wm.station.commands.
13. El Engine correspondiente consume la orden y comprueba que puede iniciar el servicio.
14. Si la acepta, prepara y guarda el estado del servicio, inicia la simulación y publica WATERING_STARTED.
15. Central registra el inicio, actualiza el estado a WATERING y reenvía la confirmación al FO.
16. Si Engine rechaza el inicio, publica WATERING_FINISHED con motivo ENGINE_KO; Central procesa el resultado y libera la reserva.

7.5.Durante el riego

1. El Engine mantiene abierta la válvula.
2. Calcula el caudal cada segundo.
3. Incrementa el tiempo de servicio y acumula el volumen suministrado.
4. Guarda el estado recuperable del servicio en su volumen persistente.
5. Publica periódicamente WATERING_TELEMETRY en wm.station.telemetry.
6. Central valida el evento y lo relaciona con la sesión.
7. Guarda la muestra de telemetría en SQLite.
8. Actualiza los últimos valores de caudal y volumen de la estación.
9. Reenvía la telemetría al topic del operario asociado al servicio.
10. El panel de Central y la aplicación FO muestran la información recibida.
11. Mientras tanto, el Monitor continúa comprobando la salud del Engine e informando a Central.

7.6.Finalización del riego

El riego termina por:

- TIME_LIMIT: se alcanzó la duración solicitada.
- MANUAL_STOP: se ejecutó la parada manual.
- LEAK: se detectó una avería o fuga.
- BLOCKED: Central bloqueó la estación.
- ENGINE_KO: el Engine no pudo iniciar o mantener el servicio.
- ENGINE_TIMEOUT: el Engine no respondió.

La finalización sigue estos pasos:

1. El Engine cierra la válvula y detiene la simulación del suministro.
2. Se crea la trama WATERING_FINISHED con los identificadores del servicio, duración, volumen y motivo.
3. Guarda el resultado pendiente en su estado persistente.

4. Publica el resultado en wm.station.results.
5. Central comprueba la sesión y si el resultado ya estaba registrado.
6. Guarda el resumen y actualiza el estado de la solicitud y de la sesión.
7. Libera la reserva de la estación.
8. Recalcula su estado visible según conexión, bloqueo y salud.
9. Reenvía el resumen al FO correspondiente.
10. El FO guarda y muestra el resultado recibido.

7.7. Detección de averías y recuperación de Engine

1. El Monitor envía una comprobación de salud a Engine.
2. Si recibe KO o falla la comunicación, identifica la incidencia.
3. Envía INCIDENT a Central mediante el socket.
4. Central registra la incidencia y recalcula el estado de la estación.
5. Si existe un servicio activo, obtiene de SQLite el operario y los identificadores asociados.
6. Publica un evento JSON INCIDENT en el topic de ese operario.
7. El FO muestra el aviso sin considerarlo un resultado final.
8. Si la avería es simulada y Engine sigue funcionando, este detiene el servicio y prepara su resumen.
9. Si Engine ha caído, al reiniciarse recupera los últimos datos guardados y prepara el resultado interrumpido.
10. Engine publica WATERING_FINISHED cuando puede hacerlo, sin reanudar automáticamente el riego anterior.
11. Cuando el Monitor vuelve a recibir una respuesta de salud correcta, comunica RECOVERY a Central.
12. Central cierra la incidencia y recalcula el estado, respetando los bloqueos y las condiciones que todavía existan.

7.8. Control desde Central y desde la estación

1. Desde el panel de Central se puede seleccionar una estación o todas las registradas y activas.
2. Para iniciar un riego, Central publica una solicitud.
3. Para bloquear, registra la intención de bloqueo y publica BLOCK_STATION.
4. Engine recibe el bloqueo, detiene el riego si lo hubiera y prepara un resultado con motivo BLOCKED.
5. Central representa la estación como OUT_OF_SERVICE cuando corresponde según la prioridad de estados.
6. Para activar, Central retira el bloqueo y publica ACTIVATE_STATION.
7. Engine retira su bloqueo local y la estación vuelve al estado que corresponda a su salud y conexión.
8. Desde el menú de Engine también se puede solicitar un riego, detenerlo, simular una avería, repararla y consultar el estado.
9. Las solicitudes iniciadas desde ese menú se publican en Kafka y requieren autorización de Central.

10. Si Engine ha perdido la conexión con el Monitor, no permite solicitar nuevos servicios desde su menú.

7.9. Ejecución de servicios desde fichero

1. El operario introduce en la web del FO la ruta del fichero, por ejemplo /servicios/riegos.txt.
2. El FO comprueba la ruta y valida el formato de sus líneas.
3. Cada línea indica una estación y una duración, separadas por ;.
4. El FO genera una solicitud para el primer servicio y la publica en Kafka.
5. Espera a recibir WATERING_FINISHED o WATERING_DENIED.
6. Tras la conclusión del servicio, espera cuatro segundos.
7. Genera y envía la siguiente solicitud.
8. Repite el proceso hasta terminar el fichero o producirse un error que interrumpa su ejecución.
9. Un aviso INCIDENT no hace avanzar por sí solo al siguiente servicio: se espera el resultado final.
10. La recepción de eventos continúa en paralelo mientras se ejecuta el fichero.

7.10. Administración de identidades y consulta de logs

1. Desde /altas se selecciona el tipo de registro: estación u operario.
2. Para dar de alta, se introduce el ID y su ubicación o nombre.
3. Central crea el registro o reactiva uno existente, conservando su historial.
4. Para dar de baja, comprueba que no haya un servicio pendiente o en curso; en una estación también exige que su Monitor esté desconectado.
5. La baja marca el ID como inactivo, sin borrar sus servicios anteriores ni eliminar sus contenedores.
6. El panel operativo deja de mostrar las estaciones dadas de baja.
7. El botón «Listar / actualizar registros» consulta los IDs y sustituye las filas de la tabla con los datos actuales.
8. El listado distingue habilitación administrativa de conexión: un ID activo no implica que su proceso esté ejecutándose.
9. El panel principal de Central incorpora una ventana con sus logs recientes.
10. Las webs de Central y FO actualizan sus ventanas de mensajes periódicamente, con altura fija y scroll para revisar mensajes anteriores.
11. Los heartbeats rutinarios se procesan sin imprimirlos continuamente en la consola de Central.

8.Despliegue previsto

8.1.Distribución de los componentes

El despliegue de Water Management Network separa la infraestructura central de los componentes ejecutados en los equipos del laboratorio.
En la parte Cloud se alojan:

- WM_Central, que gestiona las solicitudes, las estaciones y el panel web.
- SQLite, como base de datos de Central.
- Apache Kafka, como intermediario de las comunicaciones basadas en eventos.

En el laboratorio se ejecutan las aplicaciones de operario y las estaciones de riego. Cada estación está formada por dos procesos independientes: Monitor y Engine.

Cloud
├── WM_Central
├── SQLite
└── Apache Kafka

Equipo de laboratorio 1
├── WM_FO OP-01
├── WM_FO OP-02
└──WM_FO OP-N (Numero indefinido de operarios)

Equipo de laboratorio 2
├── WS-01
│ ├── WM_WS_M
│ └── WM_WS_E
├── WS-02
│ ├── WM_WS_M
│ └── WM_WS_E
└── WS-N (Numero indefinido de WS)
├── WM_WS_M
└── WM_WS_E

Todos los módulos se despliegan mediante Docker.

8.2.Requisitos previos

Para ejecutar el escenario se necesita:

- Docker instalado y en ejecución en los equipos del laboratorio.
- Acceso al repositorio del proyecto.
- Conectividad con los servicios Cloud de Central y Kafka.

- Identificadores de estaciones y operarios registrados en SQLite.
- Puertos locales disponibles para los Monitores.
- Las imágenes Docker de los módulos preparadas para su ejecución.

Antes de iniciar los módulos del laboratorio se comprueba que Central y Kafka están disponibles y que se puede acceder al panel web.

8.3. Configuración de los módulos

Parámetro	Función
CENTRAL_PORT	5000 interno en Central
WEB_PORT	8080 interno en Central.
KAFKA_BOOTSTRAP_SERVERS	Endpoint interno apropiado en Cloud; broker:29092 en Compose local
DB_PATH	Ruta del fichero SQLite.
FO_WEB_PORT	8081 dentro de cada contenedor FO
FO_CONSOLE	false para uso web; true permite el menú cuando se proporciona terminal.
ENGINE_DATA_DIR	/data como ruta predeterminada del estado de Engine.

Conexiones públicas del despliegue:

- Panel: https://central-production-8c7f.up.railway.app
- Sockets Central: switchyard.proxy.rlwy.net:16591
- Kafka externo: centerbeam.proxy.rlwy.net:45491

El Monitor recibe el puerto de escucha del Engine, la dirección y el puerto de Central y el identificador de la estación, el Engine recibe la dirección y el puerto del Monitor, el identificador de la estación y la dirección de Kafka, y la aplicación FO recibe la dirección de Kafka y el identificador del operario.

Cuando se ejecutan varias estaciones en el mismo equipo, cada Monitor utiliza un puerto diferente. Asimismo, cada instancia debe emplear el identificador de la estación u operario que representa.

8.4.Preparación de Kafka

Los topics utilizados por el sistema son:

- wm.watering.requests
- wm.fo.queries
- wm.station.commands

- wm.station.telemetry
- wm.station.results

9.Decisiones de diseño

9.1. Separación entre supervisión y operación

Se utilizan sockets TCP para el registro de las estaciones y la comprobación de salud. Kafka se emplea para las solicitudes, órdenes, telemetría y resultados.

Esta separación distingue la disponibilidad de los componentes de las operaciones de riego. El Monitor supervisa al Engine, mientras que las órdenes de negocio llegan al Engine a través de Kafka.

9.2. División de la estación en Monitor y Engine

Cada estación dispone de dos procesos independientes.

El Monitor se encarga de la comunicación con Central y de la supervisión del Engine. El Engine simula la válvula, el caudal y el volumen suministrado.

Esta división permite observar y tratar por separado la caída de cada componente.

9.3. Autorización desde Central

Central decide si una solicitud puede convertirse en un riego. Antes de autorizarla comprueba la existencia del operario y de la estación, su disponibilidad, el bloqueo y la ausencia de una reserva incompatible.

Los inicios desde FO, Central o el menú de la estación deben respetar estas comprobaciones. La apertura de la válvula no se considera autorizada por el simple hecho de publicar una solicitud.

9.4. Identificación de solicitudes, sesiones y eventos

Se utilizan identificadores distintos para representar conceptos diferentes:

- requestId: solicitud de riego.
- sessionId: servicio autorizado.
- eventId: evento Kafka.
- messageId: mensaje de socket.

La separación permite relacionar los mensajes de un servicio y reconocer reenvíos sin crear una nueva sesión.

9.5. Reserva y confirmación de inicio

Una autorización reserva la estación, pero no significa que el Engine haya comenzado a regar.

El estado WATERING se muestra cuando se recibe la confirmación de inicio. Esta distinción evita representar como activo un servicio que todavía no ha abierto la válvula.

9.6. Eventos dirigidos a cada operario

Cada operario consume su propio topic de eventos: wm.fo.<operatorId>.events

Central reenvía la información al operario asociado al servicio. La consulta de estaciones disponibles se trata como una operación distinta de la recepción de eventos de un riego.

9.7. Persistencia centralizada

SQLite almacena la información persistente de Central. Los datos generales de las estaciones se separan de su estado operativo, y las solicitudes se distinguen de las sesiones y de los resultados.

Al arrancar, Central no presupone que una estación guardada esté conectada: espera a que su Monitor vuelva a comunicar el estado real.

9.8. Panel web

El panel presenta el estado de las estaciones, su ubicación y los datos del riego. La información se obtiene periódicamente del endpoint de Central.

Los colores permiten identificar rápidamente estaciones disponibles, regando, averiadas, bloqueadas o desconectadas.

10. Resiliencia y recuperación

10.1. Criterio de recuperación

El sistema se diseña para que el fallo de un componente afecte únicamente a los servicios que dependen de él. La recuperación debe requerir reiniciar el menor número posible de módulos.

Se distinguen dos situaciones:

- Fallo de comunicación, cuando un proceso sigue funcionando pero pierde una conexión.
- Caída del proceso, cuando desaparecen también sus datos mantenidos únicamente en memoria.

Los mecanismos concretos implementados y sus límites se validan mediante las pruebas del apartado 11.

10.2. Caída de Central

Los riegos ya autorizados pueden continuar en el Engine hasta finalizar. Mientras Central no está disponible, no deben comenzar nuevos servicios sin autorización.

El Monitor intenta restablecer su conexión y repetir el registro sin cerrar la conexión que mantiene con el Engine. Al recuperarse, Central parte de la información almacenada y procesa los eventos pendientes.

El objetivo de esta recuperación es evitar que sea necesario reiniciar las estaciones o los operarios por una caída de Central.

10.3. Caída del Monitor

Cuando Central deja de recibir mensajes del Monitor, muestra la estación como DISCONNECTED.

El Engine puede terminar el riego ya iniciado, pero no debe admitir nuevos servicios, incluidos los iniciados desde su menú.

Cuando vuelve el Monitor, Engine debe restablecer su conexión y repetir la identificación. El Monitor recupera la supervisión y comunica a Central la salud de la estación.

10.4. Caída o avería del Engine

El Monitor detecta el fallo mediante una respuesta KO o por ausencia de respuesta antes del timeout. A continuación comunica la incidencia a Central.

Una avería durante el riego obliga a detenerlo. La duración y el volumen ya suministrados deben conservarse para obtener el resultado del servicio.

La recuperación tras una caída abrupta requiere un estado persistente: no basta con conservar información en variables del proceso.

10.5. Caída del FO

La caída de un operario no detiene el riego que tiene autorizado. El Engine continúa su ejecución y Central procesa su resultado.

Al volver a conectarse, el FO utiliza el mismo identificador de operario y grupo de consumo para recibir los eventos pendientes por recibir.

La comprobación de este comportamiento debe incluir el resultado final del servicio, no únicamente la reconexión del consumidor.

10.6. Fallos de Kafka

Publicar una solicitud no equivale a recibir una autorización. Ante un error de envío se informa al usuario de que la publicación no ha podido confirmarse.

Las pruebas deben comprobar también que el procesamiento vuelve a funcionar después de recuperar Kafka, sin dejar un hilo detenido mientras el resto de la aplicación aparenta seguir disponible.

10.7. Duplicados y consistencia

Los reintentos pueden provocar la recepción repetida de una solicitud o evento. El sistema utiliza los identificadores y las restricciones de persistencia para evitar nuevas sesiones y resultados duplicados:

- requestId: permite reconocer una solicitud registrada sin crear otra sesión.
- sessionId: identifica el servicio y permite guardar su resultado final una sola vez.
- eventId: permite identificar eventos y evitar muestras de telemetría duplicadas.
- Engine: conserva las sesiones ejecutadas para no volver a abrir la válvula por una orden repetida de un servicio terminado.

Las garantías de recuperación se limitan a los casos comprobados. No se afirma ausencia absoluta de pérdida de datos sin verificar los mecanismos de persistencia y publicación utilizados.

La idempotencia del productor Kafka y las transacciones SQLite resuelven problemas diferentes. No constituyen por sí solas una transacción atómica entre la base de datos y Kafka. Una caída entre guardar una operación y publicar su respuesta requiere comprobar cómo se recupera ese punto del flujo.

11.Conclusión

En el desarrollo de la practica se ha aplicado los conceptos de un sistema distribuido a una simulación de gestión de estaciones de riego. Hemos integrado una Central, estaciones formadas por Monitor y Engine y aplicaciones de operario, separando las responsabilidades de cada módulo y permitiendo su ejecución en distintos equipos.

La combinación de sockets TCP y Apache Kafka nos ha permitido trabajar con dos formas de comunicación diferentes. Los sockets se utilizan para registrar las estaciones y comprobar su salud, mientras que Kafka permite intercambiar solicitudes, órdenes, telemetría y resultados. Durante el desarrollo hemos aprendido que enviar una solicitud no significa que el servicio esté autorizado y que una autorización tampoco confirma que el riego haya comenzado.

Otro aspecto importante ha sido la recuperación ante fallos. Hemos incorporado mecanismos de reconexión, almacenamiento persistente y control de duplicados para conservar la información de los servicios y evitar que un mensaje repetido provoque otro riego. También hemos diferenciado la caída de un componente de una pérdida de comunicación y el aviso de incidencia del resultado final del servicio. Estas medidas mejoran la resiliencia.

El uso de Docker ha facilitado preparar y ejecutar los módulos en otros ordenadores, mientras que Railway permite mantener Central, SQLite y Kafka en la nube. Las interfaces web permiten observar el funcionamiento, administrar los registros y controlar los riegos de forma sencilla.